DBMS Lab

What is SQL

SQL = Structured Query Language

=> It is a language to interact with database

TYPES OF SQL Commands

Type	Meaning	Commands
DDL	Structure	CREATE, ALTER, DROP
DML	Data	Insert, Update, delete
TCL	Transaction	COMMIT, ROLLBACK
DCL	Control	GRANT, REVOKE
DQL	Query	SELECT

① CREATE TABLE

```
CREATE TABLE student(
    id NUMBER,
    name VARCHAR2(20),
    marks NUMBER
);
```

② ALTER TABLE

```
ALTER TABLE student
ADD age NUMBER;
```

=> Adds a new row

③ DROP TABLE

```
DROP TABLE student;
```

④ TRUNCATE TABLE

```
TRUNCATE TABLE student;
```

Difference b/w DROP & TRUNCATE

DROP → deletes table structure + data

TRUNCATE → deletes only data

⑤ INSERT

```
INSERT INTO student
VALUES(1,'zain',90);
```

⑥ SELECT

```
SELECT * FROM student;
```

⑧ UPDATE

```
UPDATE student SET
marks=95 where Id=1;
```

⑨ DELETE

```
DELETE FROM student
WHERE id=1;
```

SELECT

→ Basic

```
SELECT name FROM
student;
```

→ WHERE

```
SELECT* FROM student
where marks>80;
```

→ DISTINCT

```
SELECT DISTINCT
marks FROM student;
```

→ ORDER BY

```
SELECT * FROM student
ORDER BY marks DESC;
```

CONSTRAINTS

Types of Constraints

constraint	Meaning
PRIMARY KEY	Unique + NOT NULL
NOT NULL	Cannot be empty
UNIQUE	No duplicates
CHECK	Condition
FOREIGN KEY	Link tables

PRIMARY KEY

```
id NUMBER PRIMARY KEY
```

NOT NULL

```
name VARCHAR2(20) NOT NULL
```

UNIQUE

```
email VARCHAR2(50) UNIQUE
```

CHECK

```
salary NUMBER CHECK (salary>0)
```

FOREIGN KEY

```
dept_id NUMBER REFERENCES
dept(id)
```

JOIN

↳ Used to combine data from multiple tables

3 Important joins

① INNER JOIN

```
SELECT e.name, d.dept_name
FROM emp e
JOIN dept d ON
e.dept_id=d.dept_id
```

② LEFT JOIN

```
SELECT e.name, d.deptname
FROM emp e
LEFTJOIN dept d ON
e.dept_id=d.dept_id;
```

③ RIGHT JOIN

```
SELECT e.name, d.deptname
FROM emp e
RIGHT JOIN dept d ON
e.dept_id=d.dept_id;
```

PL/SQL

PL/SQL = Procedural language extension of SQL

Structure

```
DECLARE
   -- Variables
BEGIN
   -- logic
EXCEPTION
   -- error handling
END;
/
```

Variables

```
DECLARE
   num NUMBER := 10;
   name VARCHAR2(20) := 'Zain';
BEGIN
   DBMS_OUTPUT.PUT_LINE(num);
   DBMS_OUTPUT.PUT_LINE(name);
END;
/
```

NOTE:

Before running any PL/SQL:

```
SET SERVER OUTPUT ON
```

IF-ELSE

```
DECLARE
   num NUMBER := 10;
BEGIN
   IF num > 0 THEN
   DBMS_OUTPUT.PUT_LINE('Positive')
ELSE
DBMS_OUTPUT.PUT_LINE('Negative')
END;
/
```

FOR LOOP

```
DECLARE
BEGIN
   FOR i IN 1..5 LOOP
   DBMS_OUTPUT.PUT_LINE(i);
   END LOOP;
END;
/
```

WHILE LOOP

```
DECLARE
   i NUMBER := 1
BEGIN
   WHILE i<=5 LOOP
   DBMS_OUTPUT.PUT_LINE(i);
      i := i+1;
   END LOOP;
END;
/
```

EVEN/ODD

```
DECLARE
 num NUMBER := 10;
BEGIN
  IF MOD(num,2)=0 THEN
     DBMS_OUTPUT.PUTLINE('Even');
  ELSE
    DBMS_OUTPUT.PUT_LINE('odd');
  END IF;
END;
/
```

Factorial

```
DECLARE
  n NUMBER := 5;
  fact NUMBER := 1;
BEGIN
  FOR i IN 1..n LOOP
    fact := fact*i
  END LOOP;
DBMS_OUTPUT.PUTLINE('Factorial'
                    ||fact);
END;
/
```

CURSORS

* cursor = pointer to rows of query

Used when

- Query returns multiple rows
- Need to process row by row

Basic Cursor Program

```
DECLARE
  CURSOR c1 IS SELECT name
  FROM Student;
  v_name student.name%TYPE;
BEGIN
  OPEN c1;
  LOOP
    FETCH c1 INTO v_name;
    EXIT WHEN c1%NOTFOUND;
  DBMS_OUTPUT.PUT_LINE(v_name)
  END LOOP;
  CLOSE c1;
END;
/
```

Exception Handling

⇓

run-time error

⇒ Ex1 Divide by Zero

```
DECLARE
  a NUMBER := 10;
  b NUMBER := 0;
  c NUMBER
BEGIN
  c := a/b;
  DBMS_OUTPUT.PUT_LINE(c);
EXCEPTION
  WHEN ZERO_DIVIDE THEN
    DBMS_OUTPUT.PUT_LINE('Error')
END;
/
```

COMMON EXCEPTIONS

Exceptions	Meaning
ZERO_DIVIDE	division by 0
NO_DATA_FOUND	no row returned
OTHERS	any error.

PROCEDURE

→ A stored block of code used to perform some task.

Ex: 1

```
CREATE OR REPLACE
PROCEDURE greet IS
BEGIN
  DBMS_OUTPUT.PUT_LINE('Hello')
END;
/
```

HOW TO RUN

```
EXEC greet;
```

Procedure with Parameters

```
CREATE OR REPLACE PROCEDURE
add_num(a NUMBER, b NUMBER)
IS
c NUMBER;
BEGIN
  c := a+b;
DBMS_OUTPUT.PUT_LINE('sum'
                    || c);
END;
/
```

HOW TO RUN

```
EXEC add_num(5, 10);
```

FUNCTION

⇒ A function must return a value.

Syntax

```
CREATE OR REPLACE
FUNCTION funcname(a NUMBER,
                  b NUMBER)
RETURN NUMBER IS
BEGIN
  RETURN a+b;
END;
/
```

Example 2 :

```
CREATE OR REPLACE FUNCTION
 square(n NUMBER)
RETURN NUMBER IS
BEGIN
    RETURN n*n;
END;
/
```

call

```
SELECT square(5) FROM dual;
```

TRIGGER

~~SET SERVER~~

```
CREATE OR REPLACE TRIGGER
 t1
BEFORE INSERT ON student
FOR EACH ROW
BEGIN
 DBMS_OUTPUT.PUTLINE('New
                      record
                      Inserted');
END;
/
```

Even/odd

```
CREATE OR REPLACE
 FUNCTION check_even(n NUMBER)
RETURN VARCHAR2 IS
BEGIN
  IF MOD(n,2)=0 THEN
     RETURN 'Even';
ELSE
  RETURN 'Odd';
  END IF;
END;
/
```

Run

```
SELECT check_even(7) FROM
dual;
```

Prevent Invalid Data

```
CREATE OR REPLACE
 TRIGGER checkmarks
BEFORE INSERT ON student
FOR EACH ROW
 BEGIN
  IF :NEW.marks<0 THEN
  RAISE_APPLICATION_ERROR
   (-2001, 'Mark can't be negative');
END IF;
END;
/
```